PROJECT REPORT

Real-Time UPI Transaction & Fraud Detection Pipeline

A Data Engineering Project using Databricks

SUBMITTED BY
Name :- Arati Sunil Thorat
College :- Sanjivani College of Engineering
StudentId :- CT_CSI_DE_1180
Domain :- Data Engineering
Date :- 09th AUG 2026

Abstract

This project, “Real-Time UPI Transaction & Fraud Detection Pipeline”, is a data engineering pipeline built entirely on Databricks to process simulated UPI (Unified Payments Interface) transaction data and identify potentially fraudulent transactions using a simple, rule-based scoring method. Since real UPI transaction data is private and not available for student projects, the data used in this project was generated using a custom data generator notebook.

The project follows a layered data pipeline approach, commonly known as the Medallion Architecture, where data moves through Landing, Bronze, Silver and Gold stages, becoming cleaner and more structured at every step. After the data reaches the Gold layer, a fraud detection step calculates a fraud score for every transaction based on rules like transaction amount, transaction velocity (how many transactions happen in a short time) and suspicious IP addresses. Each transaction is then classified as LOW, MEDIUM or HIGH risk.

The final output of the project is a Databricks dashboard named “Real-Time UPI Transaction & Fraud Detection Pipeline”, which shows key numbers (KPIs) such as total transactions, fraud rate, critical fraud alerts and total transaction amount, along with charts and a table of high-risk transactions. During development, a real error was faced (an unresolved “timestamp” column) and fixed by using the correct column name, event_timestamp. This report documents every step of the project in the order it was actually performed, along with the outputs, screenshots and the issues faced along the way.

1. Introduction

1.1 What is Data Engineering

Data engineering is the field of building systems and pipelines that collect, clean, store and organize data so that it can be used for analysis and decision-making. In simple words, a data engineer builds the “pipes” through which raw, messy data flows in and clean, usable data comes out at the other end. Tools like Databricks, Apache Spark and Delta Lake are commonly used in the industry to build such pipelines at scale.

1.2 What is UPI and UPI Transaction Data

UPI (Unified Payments Interface) is a real-time payment system widely used in India to transfer money instantly between bank accounts using a mobile app. Every UPI transaction generates data such as a transaction ID, sender and receiver UPI IDs, bank name, amount, status (success, failed, timeout) and the time at which it happened. This kind of transaction data is extremely useful for monitoring the health of the payment system and for spotting fraud.

1.3 Need for Fraud Monitoring

As digital payments have grown, so has the risk of fraudulent transactions, such as unusually large transfers, many transactions happening very quickly from the same account, or transactions coming from suspicious IP addresses. Manually checking every transaction is not possible at scale, so a data pipeline that automatically scores every transaction for risk and presents the results on a dashboard is very useful for monitoring teams.

2. Problem Statement

Financial systems that process UPI transactions need a way to automatically flag suspicious transactions instead of relying on manual checking. This project addresses the problem of building an end-to-end pipeline that takes raw UPI transaction data, cleans and organizes it, calculates a fraud risk score for each transaction using simple rules, and presents the results through an easy-to-read dashboard so that monitoring becomes fast and visual instead of manual.

3. Objectives

- To build a complete data pipeline on Databricks using the Bronze-Silver-Gold (Medallion) architecture.
- To generate simulated UPI transaction data since real transaction data could not be used.
- To clean and transform the raw transaction data into a structured, analysis-ready format.
- To implement a rule-based fraud detection mechanism using fields like amount, transaction velocity, and IP address.
- To validate the fraud detection output using SQL queries.

- To build a Databricks dashboard that presents the fraud monitoring results through KPIs, charts and tables.

4. Technologies and Tools Used

Only the technologies actually used and visible in the project are listed below:

- Databricks — the main platform used to write notebooks, run the pipeline, and build the dashboard.
- Python — used to write the data generator and the transformation logic.
- PySpark / Spark SQL — used for reading, cleaning and transforming the transaction data at each pipeline stage.
- SQL — used to query the Gold tables and validate the fraud detection results.
- Databricks Dashboards — used to build the final "UPI Transaction Fraud Monitoring" dashboard.
- Databricks Genie Code — used to review and correct dashboard queries during development.

5. System Architecture / Data Pipeline

The project follows a step-by-step pipeline where data becomes cleaner and more useful at every stage. The overall flow is shown below:

Data Generator → Landing / Ingestion → Bronze Processing → Silver Transformation → Gold Aggregation → Fraud Detection → Validation → Dashboard / Reporting

Each stage of the pipeline was built as a separate Databricks notebook, which kept the project organised and made it easy to test and fix each stage on its own. The notebooks used in this project are: 01_Data_Generator, 02_Landing_Ingestion, 03_Bronze_Processing, 04_Silver_Transformation, 05_Gold_Aggregation, and 06_Fraud_Detection.

6. Step-by-Step Implementation

6.1 Step 1 – Data Generation (01_Data_Generator)

Since real UPI transaction data is private and cannot be used for a student project, the first step was to generate simulated UPI transaction data using Python inside the 01_Data_Generator notebook. This notebook created transaction records with fields such as transaction ID, timestamp, sender and receiver UPI IDs, bank name, amount, status and IP address, and wrote them as JSON files. This step formed the base data on which the rest of the pipeline was built.

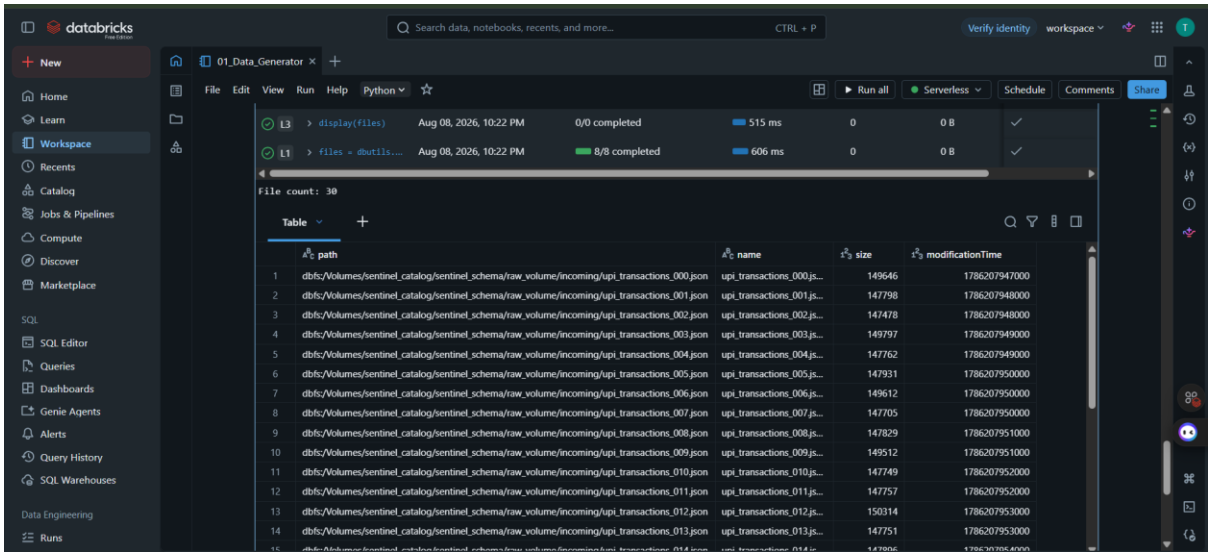


Figure 6.1: Data Generator notebook (01_Data_Generator) and generated output

6.2 Step 2 – Landing / Ingestion (02_Landing_Ingestion)

In this step, the raw JSON files created by the generator were read and copied into a Landing area without changing anything in them. The purpose of this step was to keep an untouched, raw copy of the original data before any cleaning was done, so that the source data would always be available if something needed to be re-checked later.

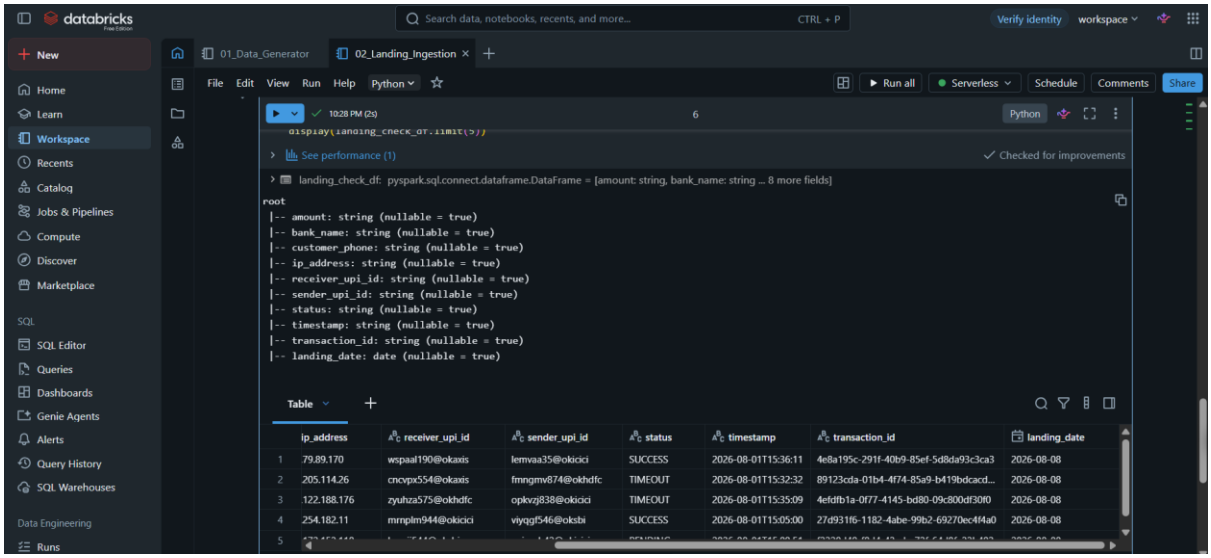


Figure 6.2: Landing / Ingestion notebook (02_Landing_Ingestion)

6.3 Step 3 – Bronze Processing (03_Bronze_Processing)

The Bronze step took the landed raw data and loaded it into a Delta table, adding extra tracking information such as the time the data was ingested. At this stage, the data was still in its raw form (including any bad or messy values); the goal was simply to store it reliably in a structured Delta table format so it could be processed further.

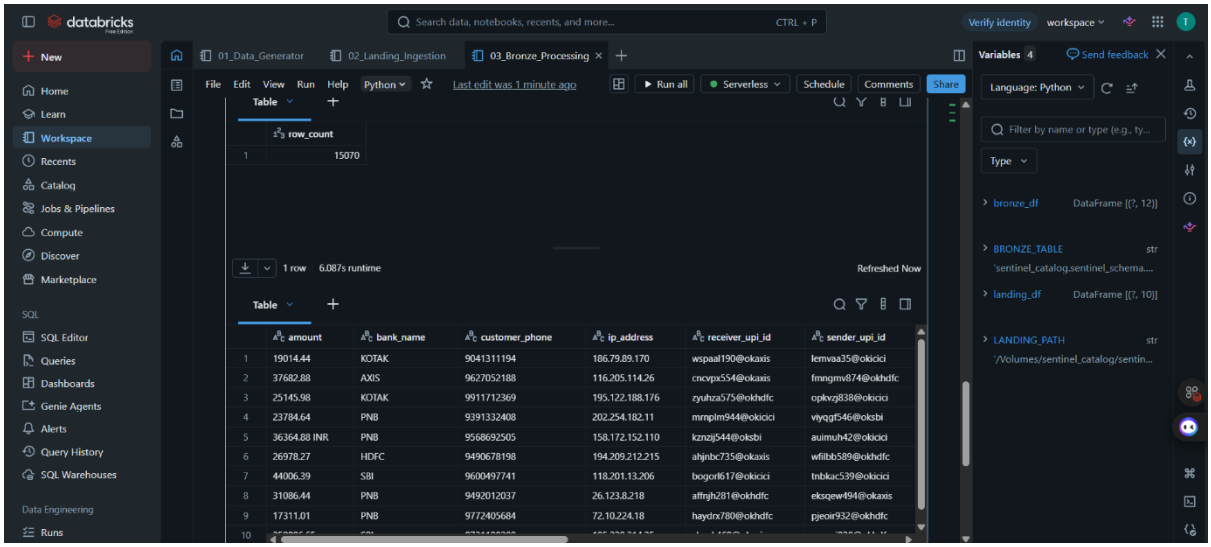


Figure 6.3: Bronze Processing notebook (03_Bronze_Processing) and Bronze table output

6.4 Step 4 – Silver Transformation (04_Silver_Transformation)

This step cleaned and standardized the Bronze data. This included fixing data types, removing extra spaces, standardizing transaction status values, and filtering out invalid records such as those with a negative or missing amount. The cleaned output was stored as the Silver table, which became the base for all later calculations and the fraud detection logic.

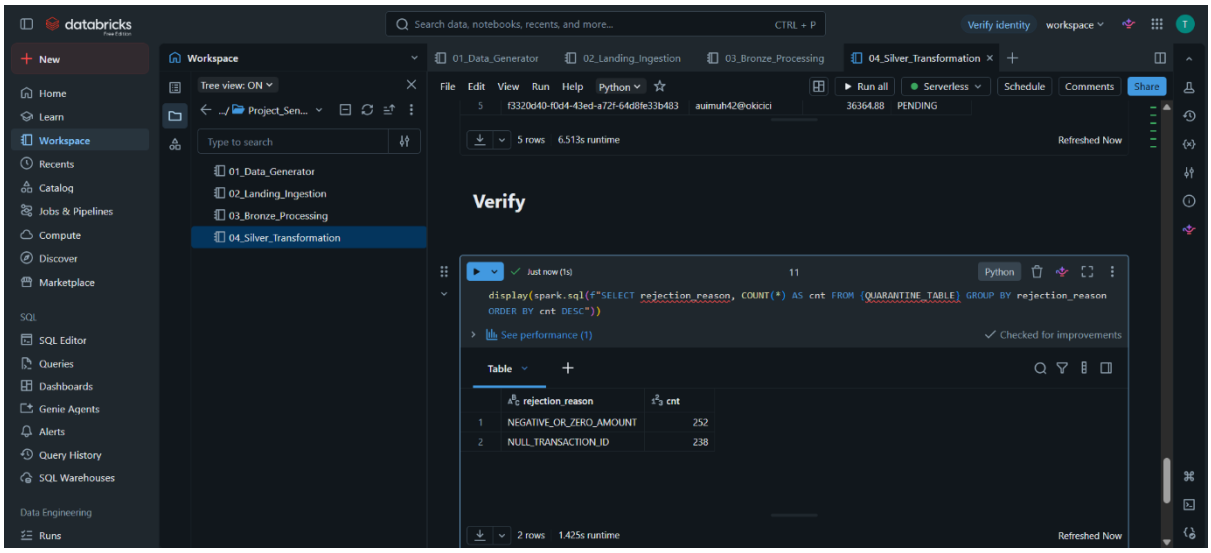


Figure 6.4: Silver Transformation notebook (04_Silver_Transformation) and Silver table output

6.5 Step 5 – Gold Aggregation (05_Gold_Aggregation)

In the Gold step, the clean Silver data was aggregated into business-level summaries, such as total transaction counts and amounts. This is the stage where the data becomes ready for direct reporting and dashboarding rather than row-by-row analysis.

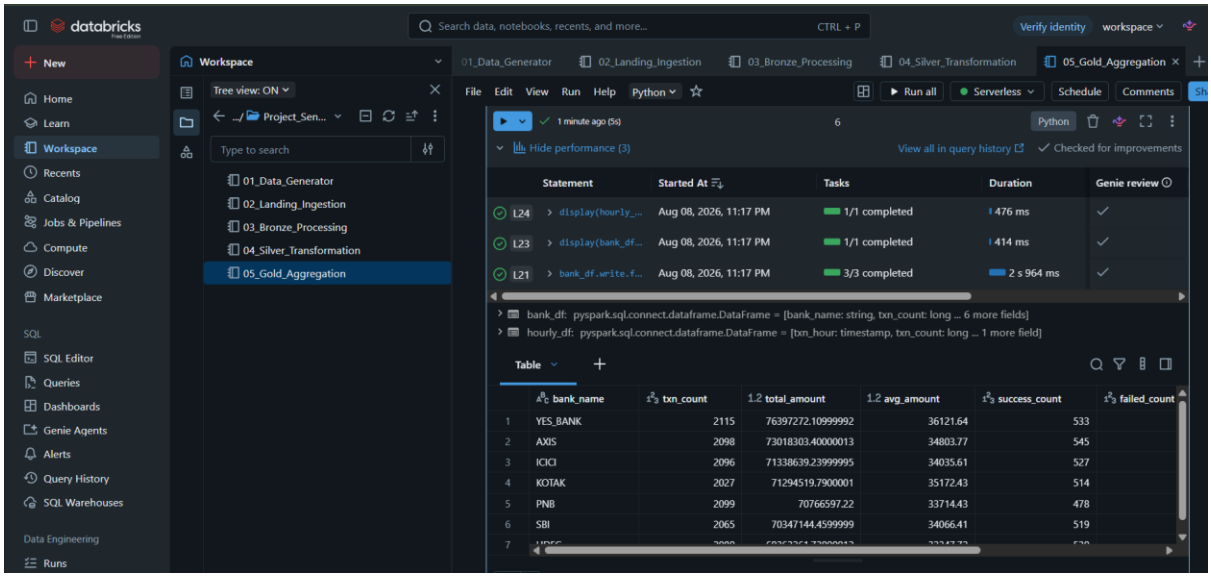


Figure 6.5: Gold Aggregation notebook (05_Gold_Aggregation) and Gold table output

6.6 Step 6 – Fraud Detection (06_Fraud_Detection)

This is the most important step of the project. In this notebook, every transaction in the Gold data was scored for fraud risk using a set of simple rules, and new columns were added: amount_rule_points, velocity_rule_points, suspicious_ip_rule_points, fraud_score and fraud_level. The complete logic behind these columns is explained in detail in Section 8 (Fraud Detection Logic) of this report.

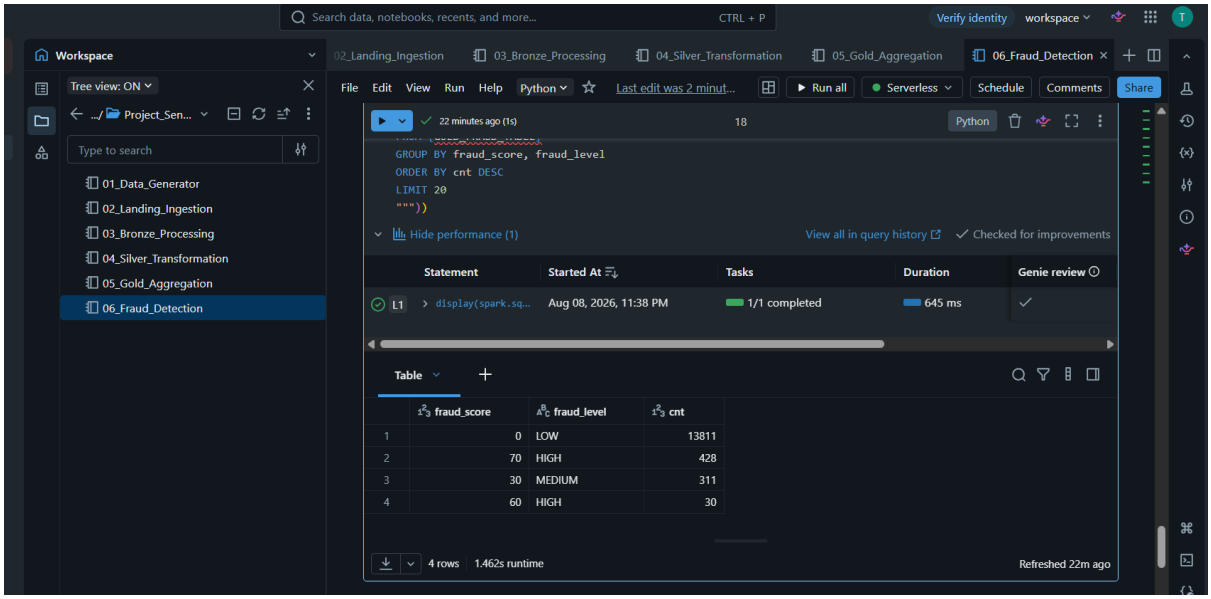


Figure 6.6: Fraud Detection notebook (06_Fraud_Detection) and fraud-scored output

6.7 Step 7 – Validation

After the fraud detection step, the results were validated by running a SQL query that grouped transactions by fraud_level and calculated the number of transactions and the average fraud score in each group. This validation confirmed that the fraud scoring logic was working as expected — HIGH

risk transactions had a high average score, while LOW risk transactions had a very low or zero average score. The exact output of this validation query is shown in Section 9 (Results and Analysis).

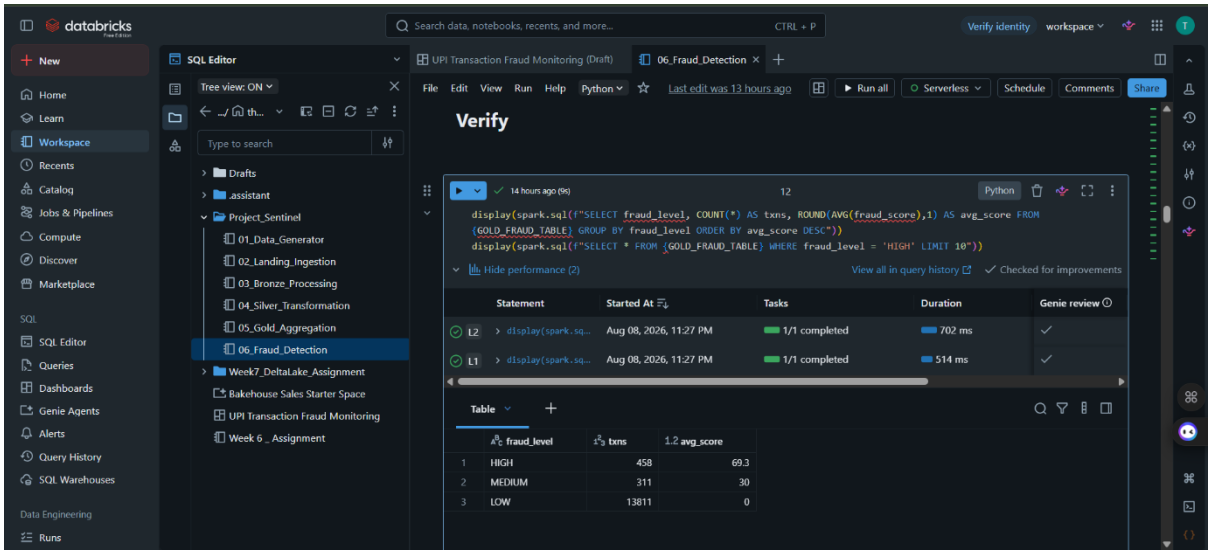


Figure 6.7: SQL validation query output showing fraud_level, txns and avg_score

6.8 Step 8 – Dashboard Creation

Once the Gold and fraud tables were validated, a Databricks dashboard named “UPI Transaction Fraud Monitoring” was created to present the results visually. The dashboard was built directly on top of the Gold fraud table using Databricks' dashboard and visualization tools, so that anyone viewing it could understand the fraud situation at a glance instead of running SQL queries manually.

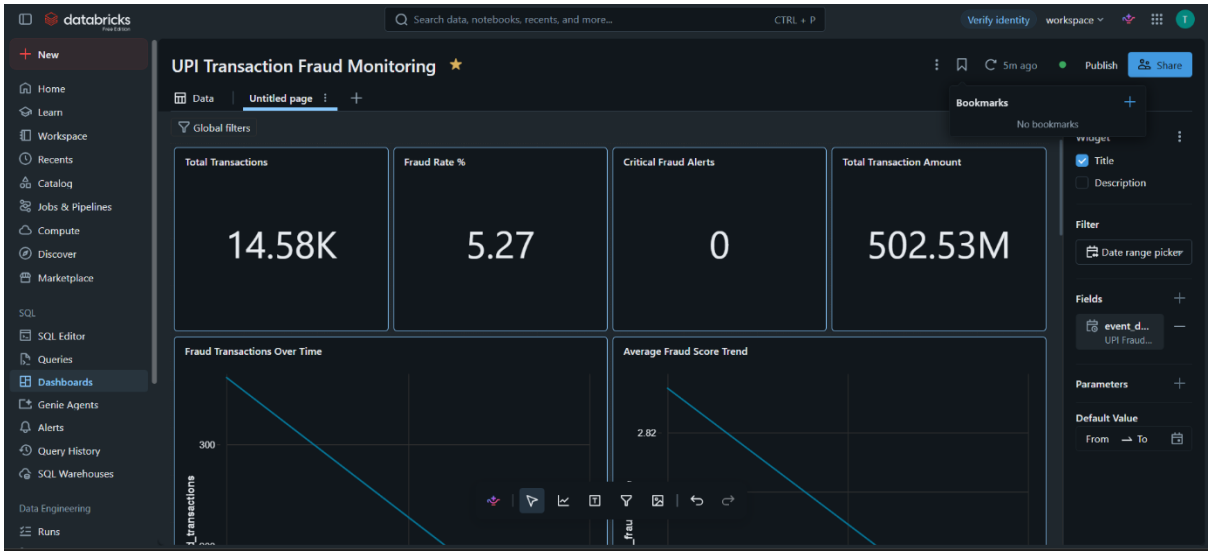
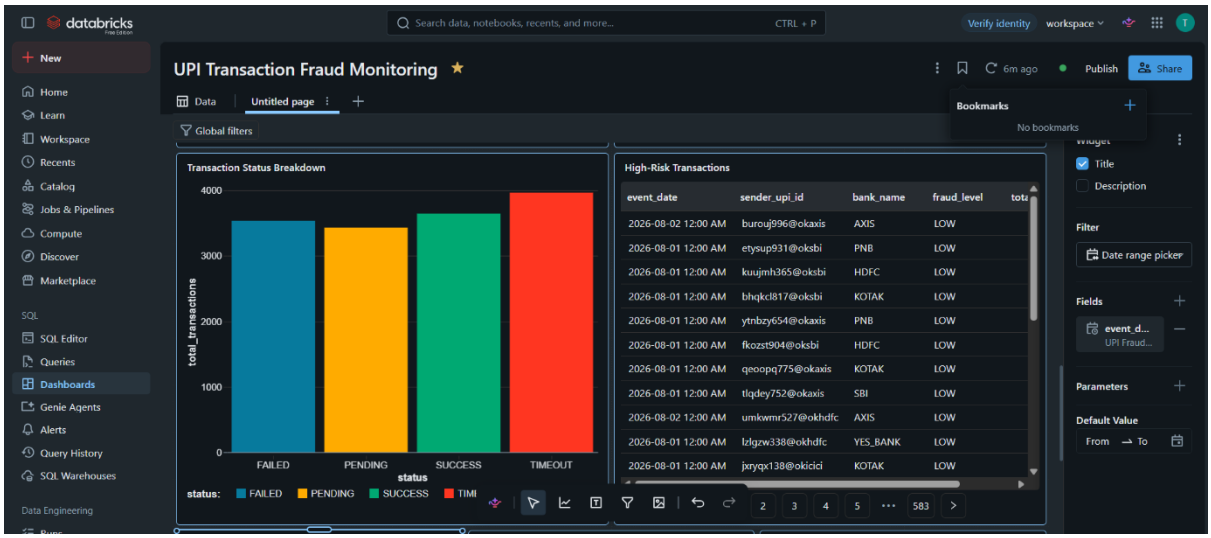


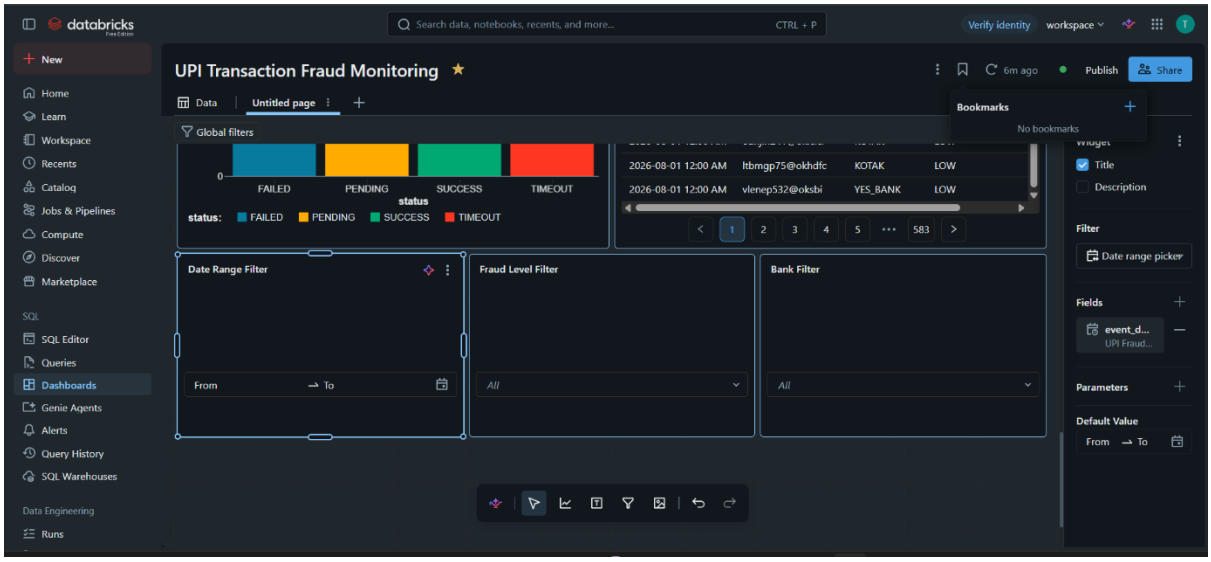
Figure 6.8: Databricks dashboard creation screen for “UPI Transaction Fraud Monitoring”

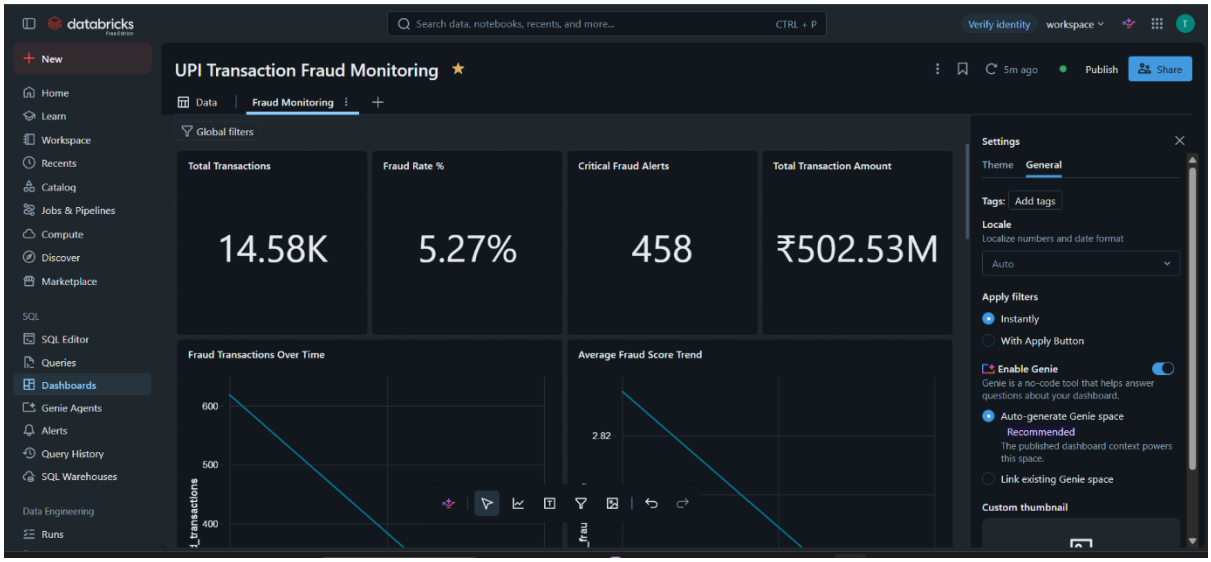
6.9 Step 9 – Final Dashboard

After fixing the column name issue and other small problems, the dashboard was completed with four KPI cards, six visualizations and three filters. Every element of the final dashboard is explained in detail in Section 10 (Dashboard Explanation) of this report.

Figure 6.10: Final "UPI Transaction Fraud Monitoring" dashboard







6.10 Step 10 – Publishing and Sharing

Once the dashboard was finalized, it was published in Databricks so that it could be viewed directly through a shared link without opening the underlying notebooks. The dashboard was made accessible to anyone who has the published link, allowing the project results and fraud monitoring visualizations to be viewed easily. The published dashboard can be accessed here:

[UPI Transaction Fraud Monitoring Dashboard]

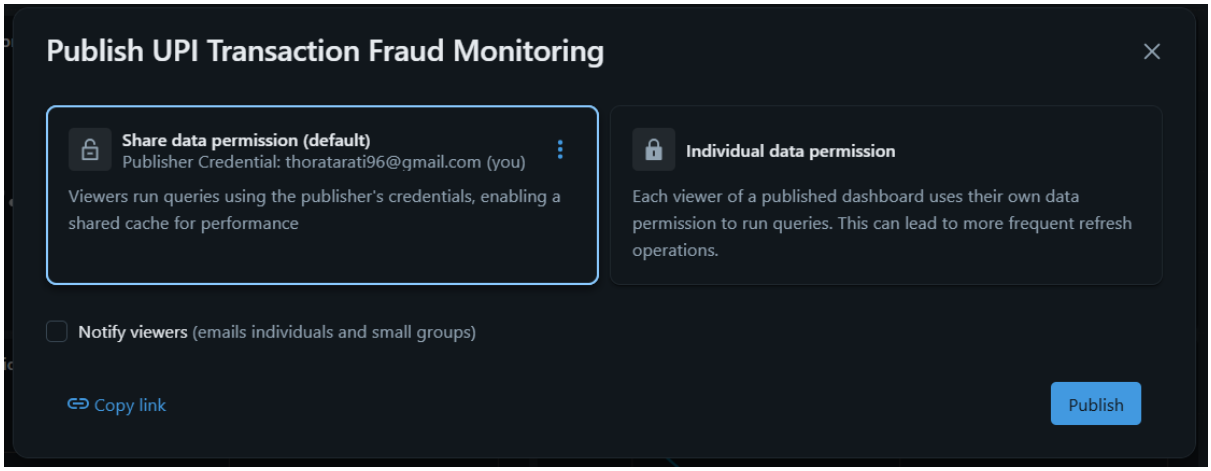


Figure 6.11: Dashboard publishing and sharing settings

7. Fraud Detection Logic

The fraud detection step does not use machine learning. Instead, it uses a simple, rule-based points system that is easy to explain and verify, which is well suited to a student project. Every transaction earns points from three separate rules, and the points are added together to get a final fraud_score:

- amount_rule_points – points added when a transaction amount is unusually high compared to typical UPI transactions.
- velocity_rule_points – points added when the same sender makes an unusually large number of transactions within a short time window, which can indicate an automated or fraudulent pattern.
- suspicious_ip_rule_points – points added when a transaction comes from an IP address that is flagged as suspicious, or when the same IP address is used by many different senders in a short time.

These three point values are added together to get the fraud_score for each transaction. Based on the fraud_score, every transaction is then classified into a fraud_level:

- LOW – little to no fraud indicators found; a normal transaction.
- MEDIUM – some fraud indicators found; the transaction is worth a closer look.
- HIGH – multiple fraud indicators found; the transaction is treated as a critical fraud alert.

This rule-based approach keeps the logic transparent — anyone reviewing a flagged transaction can see exactly which rule(s) triggered the score, instead of relying on a “black box” model.

8. Dashboard Explanation

The final dashboard, “UPI Transaction Fraud Monitoring”, is built to give a quick, visual summary of the fraud situation across all transactions. Each part of the dashboard is explained below.

8.1 KPI Cards

- Total Transactions — shows the overall number of transactions processed by the pipeline (14.58K in the final run). This gives a quick sense of the scale of data being monitored.
- Fraud Rate % — shows what percentage of all transactions were flagged as fraud-risk (5.27% in the final run). This is a quick health indicator — the lower this number, the healthier the transaction environment.
- Critical Fraud Alerts — shows the count of transactions classified as HIGH risk (458 in the final run). These are the transactions that most urgently need manual review.
- Total Transaction Amount — shows the total money value of all transactions processed, formatted in Indian Rupees (₹502.53M in the final run). This gives a sense of the financial scale being monitored, not just the transaction count.

8.2 Visualizations

- Fraud Transactions Over Time — a time-based chart showing how the number of fraud-flagged transactions changes over the date range, helping to spot spikes on particular days or times.
- Average Fraud Score Trend — a chart showing how the average fraud_score changes over time, useful for noticing if transactions are gradually becoming riskier.
- Fraud by Bank — a chart breaking down fraud counts or fraud rate by bank_name, useful for spotting if a particular bank has an unusually high share of risky transactions.
- Fraud Level Distribution — a chart showing the split between LOW, MEDIUM and HIGH risk transactions, giving an overall picture of the risk profile of the transaction data.
- Transaction Status Breakdown — a chart showing the split of transactions by status (such as success, failed, timeout), which helps separate genuine failures from fraud-related issues.
- High-Risk Transactions table — a detailed table listing only the HIGH fraud_level transactions, sorted by fraud_score, so that the most suspicious transactions are visible at the top for quick review.

8.3 Filters

The dashboard includes three filters — Date Range, Fraud Level and Bank — which let a user narrow down the data being displayed without editing any query. For example, a user can filter by Date Range to check fraud activity for a specific week, filter by Fraud Level to look only at HIGH risk transactions, or filter by Bank to check the fraud pattern for one particular bank. All the KPI cards and charts on the dashboard update automatically based on the filters selected, which makes the dashboard flexible for different kinds of monitoring questions without needing to write new SQL queries each time.

9. Results and Analysis

The final results of the project, taken directly from the dashboard and the validation query, are summarized below.

KPI	Value
Total Transactions	14.58K
Fraud Rate	5.27%
Critical Fraud Alerts (HIGH)	458
Total Transaction Amount	₹502.53M

The fraud validation query, which grouped transactions by fraud_level, produced the following result:

fraud_level	txns	avg_score
HIGH	458	69.3
MEDIUM	311	30
LOW	13811	0

In simple terms, this result means that out of all the transactions processed, 458 transactions were serious enough to be marked HIGH risk, and on average these transactions scored 69.3 points out of the fraud rule system — meaning most of them triggered more than one fraud rule at once. 311 transactions were marked MEDIUM risk with an average score of 30, meaning they usually triggered exactly one fraud rule. The vast majority, 13,811 transactions, were marked LOW risk with an average score of 0, meaning they did not trigger any fraud rule at all and can be treated as normal, safe transactions. This distribution makes sense for a monitoring system — most transactions are normal, and only a small percentage genuinely need attention, which matches the 5.27% fraud rate shown on the dashboard.

10. Challenges and Solutions

10.1 Unresolved “timestamp” Column Error

Problem: While building a chart on the dashboard, the column “timestamp” was used, but Databricks returned an unresolved column error, meaning the query engine could not find a column by that name in the table.

Solution: On checking the actual Gold table schema, the correct column name was found to be event_timestamp. Updating the chart/query to use event_timestamp instead of timestamp fixed the error, and the chart started working correctly.

10.2 Dashboard Query and Visualization Corrections

Apart from the timestamp issue, several other corrections were needed while building the dashboard. These were reviewed and fixed with the help of Databricks Genie Code, which helped check and correct the SQL logic behind each widget:

- The Fraud Rate calculation was corrected so it accurately reflects the percentage of fraud-flagged transactions out of the total.
- The Critical Fraud Alerts KPI was corrected to count only transactions where fraud_level equals HIGH.
- The KPI cards were updated so that they respond correctly to the dashboard filters (Date Range, Fraud Level, Bank) instead of always showing the full, unfiltered numbers.
- The Fraud Level Distribution chart was corrected to show the right grouping and counts.
- The High-Risk Transactions table was corrected to show only HIGH fraud_level records, sorted by fraud_score in descending order.
- Several chart queries were corrected to use the right column references matching the actual Gold table schema.
- The Total Transaction Amount KPI was formatted to display as Indian Rupee currency (₹) instead of a plain number.
- Each dashboard widget was manually re-verified against the actual table data after every fix, to make sure the numbers displayed were correct.

These corrections were a normal and expected part of dashboard development — the first version of a query or chart rarely matches the final table schema exactly, and reviewing/fixing each widget one at a time was an important part of building a dashboard that could be trusted for monitoring.

10.3 Filter Configuration

Some effort was also needed to correctly configure the Date Range, Fraud Level and Bank filters so that they were properly linked to all the relevant KPI cards and charts, ensuring the entire dashboard responds consistently when a filter is applied.

11. Final Outcome

By the end of this project, a complete data pipeline was successfully built and run on Databricks, starting from simulated UPI transaction generation, moving through Landing, Bronze, Silver and Gold layers, and finishing with rule-based fraud detection. The fraud detection output was validated using SQL and found to be working correctly. Finally, the results were published as a live, filterable Databricks dashboard named “UPI Transaction Fraud Monitoring”, giving a clear, at-a-glance view of transaction volume, fraud rate, critical alerts and transaction value, which is exactly what the project set out to achieve.

12. Conclusion

This project helped in understanding how a real-world data engineering pipeline is built step by step, from raw data generation all the way to a business-facing dashboard. Working with the Medallion Architecture (Bronze, Silver, Gold) made it clear why data engineers separate raw, cleaned and business-ready data instead of doing everything in one step. Building the rule-based fraud detection logic also helped in understanding how simple, explainable rules can be used to flag risk without needing a complex machine learning model. Finally, fixing real issues — like the unresolved timestamp column and the various dashboard query corrections — was one of the most valuable parts of the project, as it reflected the kind of debugging and verification work that is common in real data engineering roles.

13. Future Scope

The following are realistic improvements that could be made to this project in the future. These are suggestions only and were not implemented as part of the current project:

- Future enhancement: Moving from batch processing to real-time transaction streaming, so that fraud can be detected as transactions happen instead of after the fact.
- Future enhancement: Adding automated alerts (such as email or messaging notifications) whenever a HIGH risk transaction is detected.
- Future enhancement: Introducing a machine-learning-based fraud detection model in addition to the current rule-based approach, once enough labeled fraud data is available.
- Future enhancement: Adding more advanced monitoring, such as anomaly detection across multiple accounts or devices.
- Future enhancement: Scheduling the entire pipeline to run automatically at regular intervals using Databricks Workflows/Jobs, instead of running each notebook manually.